

由分层结构搜索驱动的统一自适应测试系统 (UATS) 精读笔记

原文: uats-unified-adaptive-testing-structure-search_15.pdf · 代码: <https://github.com/bigdata-ustc/UATS> · 笔记于 2026-06-27 发表于 ICML 2024 (PMLR 235), 作者 Junhao Yu, Yan Zhuang, Zhenya Huang, Qi Liu 等 (中大 + 西电)

一、解决什么问题

背景痛点。自适应测试系统 (ATS) 的目标是“用尽量少的题, 尽量准地估出学生能力”。现实里它有两大流派, 但各管各的:

- **CAT (计算机化自适应测试):** 每一步只选 1 道题, 答完一题、估一次能力、再选下一题。粒度细、估得准, 但太死板——学生一次只能面对一题, 限制了答题习惯, 且无法体现一组题之间的组合搭配。
- **MST (多阶段测试):** 每一步选一组题 (testlet), 学生在这组里自由作答。灵活, 能融入专家经验, 但题组要专家手工预先设计, 费时费力, 而且要答更多题。

一句话: CAT 强在 **adaptability (适应性)**, MST 强在 **flexibility (灵活性)**, 但二者割裂, 没有一个统一框架能同时建模、还能自动决定“每步选几道题”。

本文要解决的核心问题: 1. 能不能用一个统一框架同时表达 CAT 和 MST (CAT 是“每步 1 道”, MST 是“每步 M 道”, 本质只是每步选题数 M 不同)? 2. 能不能让框架自动从数据里学出“每步选哪些题、选几道”, 免掉专家设计题组? 3. 怎么刻画一个题组内部题目之间的相似性 / 组合效应 (别选一堆重复的题)?

本文答案: 把整个测试看成一张图——每道候选题是一条“边”, 测试就是从起点到终点搜一条最优路径/结构。这就是“测试结构搜索 (Test Structure Search)”。每步允许的最大题数 M 是个超参: $M = 1$ 退化成 CAT, M 较大就接近 MST。用一个可微的双层优化 (**bilevel optimization**) 算法直接从大规模作答数据里把这个结构学出来, 并给出收敛性 + 梯度误差的理论保证。

二、问题定义 (输入 / 输出)

符号与维度:

- 题库 V , $|V|$ 道题。每道题在 1PL-IRT 下有一个难度参数 $\beta_j \in \mathbb{R}$ (代码里叫 `question_difficulty`)。
- 学生潜在能力 $\theta \in \mathbb{R}$ (IRT 是标量; 用 NeuralCDM 时是知识概念维度的向量)。
- 测试共 T 步 (stage)。每步最多选 M 道题, 第 t 步选出的题组记 $Q_t = \{q_t^1, \dots, q_t^M\}$ 。
- 作答标签 $Y_t = \{y_t^1, \dots, y_t^M\}$, $y = 1$ 答对, $y = 0$ 答错。
- 每个学生的数据被拆成两半: **支撑集** D_t (用来估能力, 相当于“练习题”) 和 **查询集** D_u (用来检验估得准不准, 相当于“考试题”, 学生在 D_u 上的表现学生本人看不到, 只用于评估/训练选题策略)。

结构参数: 从测试节点 t 到题目槽 m 的边用 $x_q^{(t,m)} \in \{0, 1\}$ 表示, q 遍历题库再加一个“零边” (表示这个槽不选题)。最终选中 ($x = 1$) 的题组成 Q_t 。因为 0/1 离散不可微, 松弛成连续权重 $\alpha_q^{(t,m)}$, 用 Softmax 取出实际选的题。

一个具体小例子 (贴近 KT/CAT 场景):

设题库有 5 道数学题, 难度 $\beta = [-1.0, -0.3, 0.0, 0.8, 1.5]$ (负 = 简单, 正 = 难)。设 $M = 2$ (每步选 2 道, 介于 CAT 和 MST 之间), $T = 3$ (共 3 步)。- **输入:** 某学生在支撑集里已答过的记录 (开始为空)。- 第 1 步: 策略基于当前能力估计 (初始 $\theta_0 = 0$) 给 5 道题打分, 选出 2 道, 比如选了题 3 ($\beta = 0.0$) 和题 2 ($\beta = -0.3$)。学生作答 $Y_1 = \{1, 1\}$ (都对)。- 系统据此把 θ 往上调 (答对偏简单题 $\rightarrow$ 能力略高于 0)。- 第 2、3 步继续选、继续更新 θ 。- **输出:** (1) 一份为该学生定制的题序 (结构); (2) 测试结束时的能力估计 θ_T 。用查询集 D_u 上的 ACC/AUC 检验这个 θ_T 准不准。

目标就两条: **选题尽量少且贴合** (缩短测试)、 θ_T **尽量逼近真实能力**。

三、模型结构

整个 UATS 是一个 **bilevel (双层)** 结构, 外加一个把“全局搜索”变成“贪心逐步搜索”的近似。可以拆成三块:

模块 1: IRT / 认知诊断 (内层, 估能力)

输入学生当前答过的题 D_t 和题难度, 输出能力估计 θ 。用 1PL-IRT: 答对概率 $\sigma(\theta - \beta_j)$ 。通过在支撑集 D_t 上最小化交叉熵 (式 1、式 6) 把 θ 估出来。这是“内层问题”。

模块 2: 选题策略 (外层, 学 α)

对每个“题目槽”维护权重 α_q , Softmax 后就是“选每道题的概率”。策略网络 (代码 policy.py 的 MyNet) 吃进学生当前状态 + 候选题 mask + 题目难度特征, 输出 logits, 经 Softmax / Gumbel-Softmax 选题。目标是让选出来的题, 能让内层估出的 θ 在查询集 D_u 上损失最小 (式 5)。这是“外层问题”。

双层关系: 内层“给定选了哪些题 $\rightarrow$ 估能力”, 外层“调整选题 $\rightarrow$ 让能力估得更准”。外层的梯度要穿过内层的优化过程, 这正是 bilevel optimization 的难点。

模块 3: 贪心分层近似 (让它能实际跑起来)

理想的全局搜索要一次性把 T 步全规划好, 但测试是序贯的——第 $t+1$ 步答什么依赖第 t 步的作答, 未来数据拿不到, 没法全局求解。所以改成贪心逐层 (hierarchical): 在第 t 步只解“当下最该选的题”, 选完、答完、更新能力, 再走下一步。

数据流 (一步内):

$$\text{当前 } \theta_t, \alpha_t \xrightarrow{\text{Softmax/Gumbel 选题}} Q_t \xrightarrow{\text{学生作答}} Y_t \xrightarrow{\text{内层 } K \text{ 步梯度}} \theta_t^K \xrightarrow{\text{外层 1 步梯度}} \alpha_{t+1}$$

还有个工程关键点: 内层若真训到收敛 (θ^*) 开销太大, 于是只做 K 次梯度更新得到近似 θ_t^K 来代替 θ^* (式 7)。这会引入误差, 但论文用定理 4.3 证明该误差随 K 指数衰减——所以 K 取个不大的数就够。

对应代码: train.py 里 inner_algo 就是内层那 K 步 (params.inner_loop); DARTS_algo 就是外层带 Hessian 近似的一步; option_num 就是“每步额外多选几道题”, $M = \text{option_num} + 1$ 。

四、关键公式详解

公式 (1)/(6): 内层——能力估计损失

$$L(\theta, D_t) = \sum_{i=1}^t -\log p_{\theta}(Q_i, Y_i), \quad \theta_t^* = \arg \min_{\theta} L(\theta, D_t)$$

- 符号: p_{θ} 由 IRT 给出, $p = \sigma(\theta - \beta)$; 对答错的题概率取 $1 - p$ 。本质是二元交叉熵 (BCE)。
- 举例: 接第二节例子, 第 1 步选了题 3 ($\beta = 0$)、题 2 ($\beta = -0.3$), 都答对 ($y = 1$)。当前 $\theta = 0$:
 - 题 3: $p = \sigma(0 - 0) = 0.5$, 损失 $-\log 0.5 = 0.693$
 - 题 2: $p = \sigma(0 + 0.3) = \sigma(0.3) = 0.574$, 损失 $-\log 0.574 = 0.555$
 - 总损失 $L = 1.248$ 。对 θ 求导 (见式 12) $\nabla_{\theta} L = -\sum (y_i - p_i) = -[(1 - 0.5) + (1 - 0.574)] = -0.926$, 负梯度 $\rightarrow$ 沿 $+\theta$ 方向更新 $\rightarrow \theta$ 上调 (合理: 答对了, 能力该往上调)。
- 作用: 把“学生答了什么”翻译成“能力估计 θ ”, 是整个系统对学生的认知。

公式 (5): 外层——选题目标

$$\min_{\alpha} L(\theta_t^*(\alpha), D_u)$$

- 符号: 注意这里损失算在查询集 D_u 上, 而 $\theta_t^*(\alpha)$ 是用支撑集估出来的。 α 影响“选哪些题进 D_t ”, 从而间接影响 θ^* , 再影响 D_u 上的损失。
- 直觉: 好的选题, 应该让“基于这些题估出的能力”去预测学生在没见过的查询题上的表现时也很准。这就避免了“选一堆没信息量的题”。
- 作用: 这是元学习 / BOBCAT 式的训练目标——学的不是某个学生的 θ , 而是通用的选题策略 α 。

公式 (4) + Softmax: 从权重到具体选题

$$q_t^m = \arg \max_{q \in V} x_q^{(t,m)}, \quad x \approx \text{Softmax}(\alpha) : \frac{\exp(\alpha_q)}{\sum_{q'} \exp(\alpha_{q'})}$$

- 举例：某槽对 5 道题的权重 $\alpha = [0.1, 2.0, 0.5, -1.0, 0.3]$ ，Softmax 得概率约 $[0.10, 0.68, 0.15, 0.03, 0.04]$ ，于是选概率最高的题 2。训练时用 **Gumbel-Softmax** (代码 `gumbel_softmax`) 做“可微的离散采样”：前向近似 one-hot 取最大，反向梯度照常流过 Softmax (直通估计 straight-through)。
- 作用：把离散的“选哪道题”变成可微的，才能用梯度下降优化 α 。

公式 (7)：内层 K 步梯度更新

$$\theta_t^k = \theta_t^{k-1} - \gamma \nabla_{\theta} L(\theta, D_t(\alpha_t))$$

- 符号： γ 内层学习率， $k = 1..K$ 。用 θ_t^K 近似收敛解 θ_t^* 。
- 举例：接上面 $\theta = 0$ 、 $\nabla_{\theta} L = -0.926$ 、取 $\gamma = 0.1$ ： $\theta^1 = 0 - 0.1 \times (-0.926) = 0.093$ 。再算一轮梯度会更小，几步后 θ 稳定在某个正值——这就是“答对偏易题后能力上调”的数值体现。
- 作用：省算力的核心 trick。配合定理 4.3， K 不必很大。

公式 (8)：外层一步梯度更新

$$\alpha_{t+1} = \alpha_t - \beta \nabla_{\alpha} L(\theta_t^K(\alpha_t), D_u)$$

- 难点： ∇_{α} 要穿过内层那 K 步 (θ^K 是 α 的函数)，即需要对“优化过程”求导。代码用 **DARTS** 式的有限差分近似 **Hessian-向量积** (`policy.py` 的 `MyModel.update: hessian = dw - lw*(dalpha_pos - dalpha_neg)/(2*ep1)`)，避免直接算二阶 Hessian。
- 作用：真正更新选题策略的一步。

公式 (9) 定理 4.3：梯度估计误差界 (理论卖点 1)

$$\|\nabla_{\alpha} L(\theta_t^K) - \nabla_{\alpha} L(\theta_t^*)\| \leq \frac{B}{\mu} \left(L^2 (1-\gamma\mu)^K - \frac{M(\tau\mu + L\rho)}{\mu} (1-\gamma\mu)^{\frac{K+1}{2}} \right) + \frac{ML(1-\gamma\mu)^K}{\mu} + \frac{BM(\tau\mu + L\rho)}{\mu^2} (1-\gamma\mu)$$

- 读法：核心是因子 $(1-\gamma\mu)^K$ 。因为 IRT 强凸 ($\mu > 0$) 且 $\gamma \leq 1/L$ ，有 $0 < 1-\gamma\mu < 1$ ，所以 $(1-\gamma\mu)^K$ 随 K 指数衰减。
- 举例：设 $\mu = 0.2, \gamma = 1$ ，则 $1-\gamma\mu = 0.8$ 。 $K = 10$ 时 $0.8^{10} \approx 0.107$ ； $K = 30$ 时 $0.8^{30} \approx 0.0012$ 。即把内层从 10 步加到 30 步，主导误差项缩小约 90 倍。
- 作用：从理论上保证“用 θ^K 代替 θ^* ”是安全的——这是它敢省算力的底气。

公式 (10) 定理 4.4：收敛性 (理论卖点 2)

$$\frac{1}{T} \sum_{t=0}^{T-1} \|\nabla L(\theta_t^*(\alpha_t), D_u)\|^2 \leq \frac{16W}{T} (L_0 - L_T) + C$$

- 读法： T 步外层梯度的平均平方范数有上界，右边第一项随 T 以 $O(1/T)$ 衰减，第二项 C 是与 K 有关的常数 (K 越大 C 越小，证明里 $C \propto (1-\gamma\mu)^2$ 等项)。
- 作用：保证训练过程梯度会收敛、不发散。这对应实验 5.2.3——UATS 训练曲线比 BOBCAT/NCAT 更平滑稳定。

五、代码实现对照

仓库结构很精简：`model.py` (IRT/NCD + 能力估计)、`policy.py` (选题策略 + DARTS 更新)、`train.py` (双层训练主循环)、`dataset.py`、`utils/`。

论文概念 / 公式	代码位置
IRT 作答 $\sigma(\theta - \beta)$	<code>model.py: MAMLModel.compute_output (student_embed - self.question_difficulty);</code> NCD 分支用 <code>prednet_full1/2/3</code>
难度参数 β	<code>model.py: self.question_difficulty = nn.Parameter(...)</code>
内层 K 步更新 (式 7)	<code>train.py: inner_algo</code> , 循环 <code>params.inner_loop</code> 次, <code>new_params - inner_lr*grads</code>

论文概念 / 公式	代码位置
外层选题损失 (式 5) 在 D_u 上	model.py: forward 里 output_loss (用 output_labels/output_mask 即查询集), train_loss (支撑集)
Softmax / Gumbel 选题 (式 4)	policy.py: gumbel_softmax、hard_sample (straight-through)
每步选 $M = \text{option_num} + 1$ 道题、分层	policy.py: MyNet (op_num 分支, split_number 把题库分槽)、process()
题组内相似性 / 组合效应	policy.py: cal_sq_dis——用已选题难度的加权均值与候选题难度的平方距离作为打分, 鼓励选与已选题“互补/有区分”的题
外层 ∇_α 的 Hessian 近似 (式 8)	train.py: DARTS_algo + policy.py: MyModel.update (有限差分: dalpha_pos - dalpha_neg)
双层训练主循环 (算法 1)	train.py: run_unbiased_DARTS / pick_DARTS_samples / train_model
能力估计模拟评估 (5.2.2)	utils/utils.py 的 compute_auc/compute_accuracy, test_model

说明: 代码用的是 **DARTS (可微架构搜索)** 的思路来落地“结构搜索 + 双层优化”, `sampling='DARTS'` 是主方法, `random/active` 是对照。`cal_sq_dis` 是论文里“组合效应/相似性”那一笔在代码中的具体实现——它让同一题组里的题彼此尽量不冗余。

六、小结

实验结论: - **成绩预测 (vs CAT 基线)**: 三个数据集 (ASSIST / NIPS-EDU / EXAM)、IRT 与 NeuralCDM 两种诊断模型、 $T = 5/10/20$ 三种长度下, UATS 整体最好。相比原 SOTA, AUC@20 平均 +1.31%, ACC@20 +0.71%。- **vs MST 基线**: ACC 平均 +2.7%, AUC +3.1%。- **能力估计效率**: 达到相同估计误差, UATS 平均少用 **20%** 的题 (图 4a)。- **题目包数量的权衡** (图 4b): 包越多 $\rightarrow$ 越像 MST, 同步数误差越大、要答更多题; 包越少 $\rightarrow$ 越像 CAT, 适应性更强。调 M 就能在 CAT 与 MST 之间平滑过渡。- **训练稳定性** (图 4c): 训练曲线比 BOBCAT/NCAT 更平滑、波动更小 (呼应定理 4.4)。

优点: 1. 首次把 CAT 与 MST 统一进“结构搜索”一个框架, M 一个超参就能切换粒度, 免专家设计题组。2. 可微 + 双层优化, 端到端从数据学选题策略。3. 有理论保证: 梯度误差随 K 指数衰减 (定理 4.3)、训练收敛 (定理 4.4), 这是相比纯 RL (NCAT) / 纯元学习 (BOBCAT) 方法的差异化卖点——更稳。

局限 / 可吐槽点: - 理论以 **1PL-IRT 强凸** 为前提; 换成 NeuralCDM 这类非凸诊断模型时, 强凸假设不再严格成立, 定理的保证打折扣 (论文实验仍用了 NCD, 但理论只覆盖 IRT)。- “学生能力 θ^* 全程不变”是 ATS 的标准假设, 对真实学习过程 (边测边学、有遗忘) 并不成立——这与 KT 的动态建模是两个出发点。- MST 对比用的是 ATA 自动组卷模拟的题组 (真 MST 数据保密拿不到), 不是真实专家题组, 对比强度有保留。- 公平性问题论文明确划到范围外。- 代码里 `cal_sq_dis` 用的是题目难度的一维平方距离来近似“相似性/组合效应”, 比较朴素; 更细的内容/知识点层面的组合效应还有空间。